a bit clearer

C PROGRAMMING FROM SCRATCH · EPISODE 12

Data Type Ranges

limits.h, float.h and overflow

KEY POINTS

- One byte gives 256 patterns, so a signed char holds -128 to 127 and an unsigned char 0 to 255.
- More bytes, wider range; unsigned types start at 0, so their maximum is a little more than double.
- Don't memorize the limits: they are named constants in limits.h and float.h.
- Past the range, unsigned types wrap around; signed overflow is undefined behavior.

01 Where a range comes from

▶ VIDEO 00:43

One byte is 8 bits, and each extra bit doubles the number of patterns: **$2^8 = 256$**. An `unsigned char` uses them all for 0 and up (**0 … 255**); a `signed char` gives half to negative values, and zero takes a spot on the non-negative side (**-128 … 127**).

02 Ranges of the integer types

▶ VIDEO 01:29

TYPE	BYTES (VS)	SIGNED RANGE	UNSIGNED: 0 … MAX
`char`	1	-128 … 127	255
`short`	2	-32,768 … 32,767	65,535
`int`	4	-2,147,483,648 … 2,147,483,647	4,294,967,295
`long long`	8	max 9,223,372,036,854,775,807	

A `long` is 4 bytes in Visual Studio (same range as `int`) and 8 bytes with GCC on Linux (same as `long long`).

> KEY IDEA
>
> Pick a type by the biggest value it must hold: a `short` is plenty for a year or an exam score, but lifetime savings need a `long long`.

03 Floating-point ranges

▶ VIDEO 03:02

`float` reaches about 3.4×10^{38} with about 6 significant digits; `double` reaches about 1.8×10^{308} with about 15. For decimals, precision usually matters more than range.

04 Look it up: limits.h and float.h

▶ VIDEO 03:33

`SCHAR_MIN`, `SCHAR_MAX`, `SHRT_MIN` … `LLONG_MAX` live in `<limits.h>`; `FLT_MAX`, `DBL_MAX` in `<float.h>`. A leading **U** means unsigned (`UINT_MAX`), and unsigned types have only a MAX, because their minimum is always 0.

C — C SOURCE

```c
printf("INT_MAX: %d\n", INT_MAX);         // INT_MAX: 2147483647
printf("UINT_MAX: %u\n", UINT_MAX);       // UINT_MAX: 4294967295
printf("LLONG_MAX: %lld\n", LLONG_MAX);   // LLONG_MAX: 9223372036854775807
printf("DBL_MAX: %e\n", DBL_MAX);         // DBL_MAX: 1.797693e+308
```

05 Going past the edge

▶ VIDEO 05:10

C — C SOURCE

```c
unsigned char c = 255;
c = c + 1;          // c is now 0
```

Unsigned types **wrap around** like an odometer, and the C standard guarantees it. For signed types, going past the maximum (`INT_MAX + 1`) is **undefined behavior**: never let a signed value overflow.

CODE 2 files on GitHub github.com/abitclearer/c-programming-from-scratch/tree/main/12-data-type-ranges